



FILE ORGANIZER

A Python Project to Automatically Organize
Files into Organized Folders



 Automatic file categorization

 Simple and easy to use

 Error handling & validation

 Built with Python & Tkinter



Organize. Simplify. Save Time.

دستیار مدیریت فایل‌های کامپیوتر

فرض کن پوشه Downloads همیشه شلوغ.

برنامه تو:

- فایل‌ها را اسکن می‌کند .
 - عکس‌ها را داخل پوشه Images می‌ریزد .
 - PDFها را داخل Documents.
 - ویدیوها را داخل Videos.
 - فایل‌های فشرده را داخل Archives.
 - در آخر گزارش می‌دهد چند فایل جابه‌جا شده است
- (دوستان ما قراره ریز ریز و قدم به قدم چیزا رو یاد بگیریم اگر یک سری مبانی اولیه دیدید تعجب نکنید)

قدم اول: نمایش فایل‌های داخل یک پوشه

فعلاً اصلاً قرار نیست چیزی جابه‌جا کنیم. فقط می‌خواهیم پایتون بتواند محتویات یک پوشه را بفخواند.

ماژول (Module)

پایتون خودش هزاران قابلیت آماده دارد که داخل ماژول‌ها قرار گرفته‌اند. مثلاً:

- کار با فایل‌ها: os
- کار با زمان : datetime
- کار با اینترنت : requests
- کار با ریاضی : math

برای استفاده باید import کنیم:

```
import os
```

یعنی:

پایتون، ابزارهای ماژول os را در اختیار من قرار بده

ماژول os چیست؟

OS مخفف Operating System است.

این ماژول به برنامه اجازه می‌دهد با سیستم عامل صحبت کند.

مثلاً:

- پوشه بسازد
 - فایل حذف کند
 - نام فایل‌ها را بفحاند
 - مسیر فایل‌ها را پیدا کند
-

اولین کد

```
import os
files = os.listdir()
print(files)
```

خط اول

```
import os
```

ماژول را وارد می‌کنیم.

خط دوم

```
files = os.listdir()
```

اینجا دو مفهوم مهم داریم.

تابع (Function)

```
listdir()
```

یک تابع آماده در ماژول os است.

اسمش از این آمده:

```
list directory
```

یعنی:

ممتویات یک پوشه را به صورت لیست برگردان.

مقدار برگشتی (Return Value)

مثلاً اگر داخل پوشه این فایل‌ها باشند:

```
photo.jpg
```

```
movie.mp4
```

```
book.pdf
```

تابع این را برمی‌گرداند:

```
['photo.jpg', 'movie.mp4', 'book.pdf']
```

و ما داخل متغیر files ذخیره می‌کنیم.

خط سوم

```
print(files)
```

چاپ کردن مقدار متغیر.

فروبی:

```
['photo.jpg', 'movie.mp4', 'book.pdf']
```

مالا یک مفهوم مهم دیگر

لیست (List)

این:

```
['photo.jpg', 'movie.mp4', 'book.pdf']
```

یک List است.

لیست یعنی چند داده را داخل یک ظرف نگهداری کنیم.

مثال:

```
fruits = ['apple', 'banana', 'orange']
```

دسترسی به اعضا:

```
print(fruits[0])
```

فروبی:

```
apple
```

چون شماره گذاری از صفر شروع می شود.

مشکل کد فعلی

الان همه فایل ها را یکجا نشان می دهد:

```
['photo.jpg', 'movie.mp4', 'book.pdf']
```

ولی ما می خواهیم تک تک نمایش داده شوند.

برای این کار از حلقه استفاده می‌کنیم.

حلقه for

```
import os
files = os.listdir()
for file in files:
    print(file)
```

خروجی:

```
photo.jpg
movie.mp4
book.pdf
```

این حلقه دقیقاً چه می‌کند؟

فرض کن:

```
files = ['photo.jpg', 'movie.mp4', 'book.pdf']
```

پایتون پشت صحنه تقریباً این کار را می‌کند:

```
file = 'photo.jpg'
print(file)
file = 'movie.mp4'
print(file)
file = 'book.pdf'
print(file)
```

تا وقتی اعضای لیست تمام شوند.

تمرین شماره ۱

این کد را اجرا کن:

```
import os
files = os.listdir()
for file in files:
    print(file)
```

و به این سوالات جواب دهید:

۱. خروجی چه بود؟

۲. فایل‌های داخل چه پوشه‌ای را نمایش داد؟

۳. آیا می‌دانی چرا `os.listdir()` ورودی هم کار کرد؟

نکته:

وقتی می‌نویسی:

```
os.listdir()
```

پایتون به طور پیش‌فرض محتویات پوشه فعلی (Current Working Directory) را می‌خواند؛ یعنی همان پوشه‌ای که فایل پایتون از آنجا اجرا شده.

قدم دوم: کار با Path مسیر

الان برنامه فقط پوشه فعلی را می‌خواند.

ولی پروژه ما باید بتواند مثلاً پوشه Downloads را بخواند.

برای این کار باید مفهوم مسیر (Path) را یاد بگیری.

مثلاً:

C:\Users\Ali\Downloads

یا

D:\Movies

این‌ها Path هستند.

تابع `listdir` با ورودی

تابع `listdir()` می‌تواند یک مسیر بگیرد:

```
import os
files = os.listdir("C:\\Users\\Ali\\Downloads")
print(files)
```

و محتویات آن پوشه را برمی‌گرداند.

چرا دوتا بک‌اسلش؟

این موضوع خیلی مهم است.

در پایتون:

"\n" یعنی رفتن به خط بعد. و "t" یعنی Tab. به این‌ها Escape Sequence می‌گویند

پس اگر بنویسیم: "C:\new" پایتون فکر می‌کند \n یعنی خط جدید! برای همین می‌نویسیم:

"C:\\new" یا روش بهتر: r"C:\new"

مرف `r` یعنی Raw String و پایتون بک‌اسلش‌ها را دستکاری نمی‌کند.

تمرین

بین پوشه پروژه‌ها کجاست.

مثلاً:

```
D:\PythonProjects\FileManager
```

بعد این کد را اجرا کن:

```
import os
files = os.listdir(r"D:\PythonProjects\FileManager")
for file in files:
    print(file)
```

باید همان فایل‌هایی را ببینی که داخل آن پوشه هستند.

یک مفهوم خیلی مهم‌تر

فرض کن پروژه را به دوستت بدهی.

اگر داخل کد بنویسی:

```
os.listdir(r"C:\Users\Narges\Downloads")
```

روی سیستم او فراب می‌شود چون چنین مسیری ندارد.

برنامه‌نویس‌ها معمولاً مسیر را از کاربر می‌گیرند.

input

```
path = input("Enter folder path: ")
print(path)
```

اگر کاربر بنویسد: D:\Movies مقدارش داخل متغیر path ذخیره می‌شود.

ترکیبش با listdir

```
import os
```

```
path = input("Enter folder path: ")
```

```
files = os.listdir(path)
```

```
for file in files:
```

```
    print(file)
```

الان برنامه هر پوشه‌ای را که کاربر وارد کند می‌خواند.

یک سوال الان اینجا اگر کاربر توی مسیر دوتا اسلش نزنه ادرس خراب نمیشه؟

سؤال خوبیه. جوابش اینه که بستگی داره چطور رشته رو بنویسی.

اگر اینطوری بنویسی:

```
path = "C:\Users\Narges\Downloads"
```

ممکنه مشکل ایجاد بشه، چون پایتون بعضی ترکیب‌های \ را به عنوان کاراکترهای خاص تفسیر می‌کنه.

مثلاً:

```
print("Hello\nWorld")
```

خروجی:

Hello

World

چون \n یعنی رفتن به خط بعد.

برای جلوگیری از این مشکل سه راه رایج وجود داره:

۱. دوتا بک‌اسلش

```
path = "C:\\Users\\Narges\\Downloads"
```

هر \\\ به یک \\\ واقعی تبدیل میشه.

۲) Raw String. بهترین روش برای مسیرهای ویندوز

```
path = r"C:\Users\Narges\Downloads"
```

مرف r قبل از رشته یعنی: بک اسلش‌ها را دستکاری نکن. این روش خیلی تمیزتره.

۳. استفاده از اسلش معمولی /

شاید عجیب باشه ولی در پایتون روی ویندوز هم معمولاً کار می‌کنه:

```
path = "C:/Users/Narges/Downloads"
```

فیلی از برنامه‌نویس‌ها این روش را ترجیح می‌دن چون روی ویندوز و لینوکس و مک یکسانه.

اما یک نکته مهم:

وقتی مسیر را از کاربر با `input()` می‌گیری:

```
path = input("Enter path: ")
```

و کاربر می‌نویسه:

```
C:\Users\Narges\Downloads
```

دیگه لازم نیست دوتا بک اسلش داشته باشه، چون این متن مستقیماً از کاربر دریافت شده و داخل کد نوشته نشده که پایتون بخواد Escape Sequences ها را تفسیر کنه.

یعنی این کاملاً اویکه:

```
Enter path:
```

```
C:\Users\Narges\Downloads
```

مشکل فقط زمانی پیش میاد که خودت مسیر را به صورت رشته داخل کد بنویسی.

برای اینکه تفاوت رو کامل مس کنی، این دو خط را امتحان کن:

```
print("C:\new")
```

۹

```
print(r"C:\new")
```

بعد فروبی هر دو را ببین. دقیقاً متوجه می‌شی چرا Raw String وجود دارد 😊 .

قدم سوم: بررسی وجود داشتن مسیر

فرض کن کاربر این را وارد کند:

```
D:\Movies
```

همه چیز خوب است. اما اگر این را وارد کند:

```
D:\abcdefg
```

که اصلاً وجود ندارد، برنامه خطا می‌دهد.

ماژول `os.path`

قبلاً از `os` استفاده کردیم. داخل `os` یک بخش به نام `path` هم وجود دارد که برای کار با مسیرهاست. مثلاً:

```
os.path.exists()
```

این تابع بررسی می‌کند که یک مسیر وجود دارد یا نه.

مثال

```
import os
```

```
path = input("Enter path: ")  
print(os.path.exists(path))
```

اگر مسیر وجود داشته باشد:

True

اگر وجود نداشته باشد:

False

Boolean چیست؟

فروجهی `exists()` از نوع Boolean است.

در پایتون فقط دو مقدار Boolean داریم:

True

False

یعنی:

درست

نادرست

استفاده از if

حالا می‌توانیم تصمیم بگیریم.

```
import os  
path = input("Enter path: ")  
if os.path.exists(path):  
    print("Folder exists")
```

else:

```
print("Folder not found")
```

if دقیقاً چه کار می‌کند؟

if condition:

یعنی:

اگر شرط درست بود، کدهای زیر را اجرا کن. مثال:

```
age = 20
```

```
if age >= 18:
```

```
    print("Adult")
```

چون ۲۰ بزرگ‌تر از ۱۸ است، چاپ می‌شود.

Indentation تورفتگی

این یکی از مهم‌ترین قوانین پایتون است.

در پایتون:

```
if True:
```

```
    print("Hello")
```

درسته.

اما:

```
if True:
```

```
print("Hello")
```

فطاست.

چون پایتون از فاصله‌ها برای تشخیص بلوک‌های کد استفاده می‌کند.

تمرین

این کد را اجرا کن:

```
import os
path = input("Enter path: ")
if os.path.exists(path):
    print("Folder exists")
else:
    print("Folder not found")
```

یک بار مسیر درست بده.

یک بار مسیر اشتباه بده.

و فروجی هر دو را ببین.

سؤال مهم

فکر می‌کنی اگر کاربر به جای پوشه، مسیر یک فایل را بدهد:

D:\photo.jpg

تابع `exists()` چه فروجی می‌دهد؟

قبل از اجرا مدس بزن. این نکته برای مرحله بعدی پروژه خیلی مهم است، چون ما باید تفاوت فایل و پوشه را هم یاد بگیریم.

اگر فایل واقعاً وجود داشته باشد:

```
import os  
print(os.path.exists(r"D:\photo.jpg"))
```

خروجی: True می‌شود

چون exists() فقط می‌پرسد:

آیا این مسیر وجود دارد؟

برایش فرقی نمی‌کند فایل باشد یا پوشه.

برای تشخیص تفاوت فایل و پوشه باید از توابع تخصصی‌تر استفاده کنیم:

آیا فایل است؟

```
os.path.isfile(path)
```

مثال:

```
import os  
path = r"D:\photo.jpg"  
print(os.path.isfile(path))
```

خروجی: True

آیا پوشه است؟

```
os.path.isdir(path)
```

مثال:

```
import os  
path = r"D:\Downloads"  
print(os.path.isdir(path))
```


خروجی: True

چرا برای پروژه ما مهم است؟

چون پروژه ما باید فقط پوشه را بگیرد.

اگر کاربر این را وارد کند:

D:\photo.jpg

نباید سعی کنیم محتویاتش را بخوانیم، چون فایل است نه پوشه.

نسخه بهتر برنامه

```
import os
path = input("Enter folder path: ")
if not os.path.exists(path):
    print("Path does not exist")
elif not os.path.isdir(path):
    print("This is not a folder")
else:
    print("Folder found")
```

not True می‌شود False و not False می‌شود True

مثال:

```
is_student = False
if not is_student:
    print("Not a student")
```

: elif

قبلاً داشتیم:

if

else

اما گاهی چند شرط داریم:

```
if score >= 90:
```

```
    print("A")
```

```
elif score >= 80:
```

```
    print("B")
```

```
else:
```

```
    print("C")
```

یعنی:

- اگر شرط اول درست بود اجرا کن.
- وگرنه شرط دوم را چک کن.
- اگر هیچکدام درست نبودند. else.

تمرین

این کد را اجرا کن و سه حالت را تست کن:

۱. یک پوشه واقعی

۲. یک فایل واقعی

۳. یک مسیر الکی

توابع (Functions)

فرض کن ۱۰ جا از برنامه بخواهی بنویسی:

```
import os
path = input("Enter folder path: ")
if not os.path.exists(path):
    print("Path does not exist")
elif not os.path.isdir(path):
    print("This is not a folder")
else:
    print("Folder found")
```

هر بار باید این کد طولانی رو تکرار کنی.

برنامه‌نویس‌ها از توابع استفاده می‌کنن تا یک کار را یک بار بنویسن و هر وقت خواستن صداش بززن.

ساخت اولین تابع

```
def say_hello():
    print("Hello")
```

اینجا: `def` یعنی: داریم یک تابع تعریف می‌کنم.

اسم تابع `say_hello` و پرانتز () فعلاً یعنی تابع هیچ ورودی‌ای نمی‌گیرد.

اجرای تابع

تعریف کردن تابع کافی نیست. باید صداش بزنی:

```
say_hello()
```

کل برنامه:

```
def say_hello():  
    print("Hello")  
say_hello()
```

خروجی:

Hello

تابع با ورودی

مثلاً:

```
def greet(name):  
    print("Hello", name)
```

استفاده:

```
greet("Narges")
```

خروجی:

Hello Narges

برای پروژه هودمان

می‌توانیم بررسی پوشه را داخل یک تابع بگذاریم.

```
import os  
def check_folder(path):  
    if not os.path.exists(path):  
        print("Path does not exist")
```

```
elif not os.path.isdir(path):  
    print("This is not a folder")
```

```
else:  
    print("Folder found")
```

استفاده:

```
path = input("Enter folder path: ")
```

```
check_folder(path)
```

اما یک مشکل وجود دارد

الان تابع فقط چاپ می‌کند.

برنامه‌نویس‌ها معمولاً ترجیح می‌دهند تابع نتیجه را برگرداند.

اینجا وارد مفهوم return می‌شویم.

return چیست؟

مثال:

```
def add(a, b):  
    return a + b
```

استفاده:

```
result = add(5, 3)  
print(result)
```

فروچی: 8

تفاوت print و return

این:

```
def add(a, b):  
    print(a + b)
```

فقط روی صفحه نشان می‌دهد.

اما:

```
def add(a, b):  
    return a + b
```

مقدار را به برنامه برمی‌گرداند تا بعداً از آن استفاده کنیم.

برای پروژه

نسخه مرفه‌ای‌تر:

```
import os  
def is_valid_folder(path):  
    return os.path.exists(path) and os.path.isdir(path)
```

اینجا چیز جدید هم یاد گرفتیم:

and

یعنی:

هر دو شرط باید درست باشند.

استفاده:

```
path = input("Enter folder path: ")
```

```
if is_valid_folder(path):  
    print("Folder found")  
else:  
    print("Invalid folder")
```

چرا این بهتر است؟

چون بعداً می‌توانیم بارها از `is_valid_folder()` استفاده کنیم بدون اینکه دوباره کدها را بنویسیم.

این دقیقاً یکی از اصول مهم برنامه‌نویسی است:

Don't Repeat Yourself (DRY)

خودت را تکرار نکن.

تمرین

این تابع را خودت بنویس:

```
def is_valid_folder(path):  
    ...
```

و کاری کن:

- اگر مسیر یک پوشه واقعی بود `True`

- در غیر این صورت `False`

سپس با `print(is_valid_folder(path))` تستش کن.

کد ما تا اینجا پروژه :

```
import os

def is_valid_folder(path):

    return os.path.exists(path) and os.path.isdir(path)

path = input("Enter folder path: ")

if is_valid_folder(path):

    print("Folder found")

else:

    print("Invalid folder")
```

قدم چهارم: خواندن فایل‌های داخل پوشه

ما تا اینجا فقط فهمیدیم پوشه معتبر هست یا نه.

حالا می‌خواهیم محتویاتش را بخوانیم.

os.listdir()

قبلاً فیلی کوتاه دیدیم:

os.listdir(path)

این تابع تمام فایل‌ها و پوشه‌های داخل یک مسیر را به صورت یک لیست برمی‌گرداند.

مثلاً:

فرض کن داخل پوشه این‌ها باشند:

photo.jpg

movie.mp4

book.pdf

Documents

اگر بنویسی:

```
files = os.listdir(path)
```

مقدار files می‌شود:

```
['photo.jpg', 'movie.mp4', 'book.pdf', 'Documents']
```

نمایش کل لیست

```
print(files)
```

خروجی:

```
['photo.jpg', 'movie.mp4', 'book.pdf', 'Documents']
```

اما این برای ما زیاد جالب نیست.

حلقه for

ما می‌خواهیم تک‌تک اعضای لیست را بررسی کنیم.

```
for file in files:
```

```
    print(file)
```

اتفاقی که می‌افتد:

دور اول:

```
file = 'photo.jpg'
```

دور دوم:

```
file = 'movie.mp4'
```

دور سوم:

```
file = 'book.pdf'
```

و همینطور ادامه پیدا می‌کند.

نسخه کامل تا اینجا

```
import os

def is_valid_folder(path):
    return os.path.exists(path) and os.path.isdir(path)

path = input("Enter folder path: ")

if is_valid_folder(path):
    files = os.listdir(path)
    for file in files:
        print(file)
else:
    print("Invalid folder")
```

بهبودش کنیم

فقط اسم فایل‌ها را چاپ می‌کند. بیاییم شماره هم بگذاریم.

enumerate

یک تابع آماده پایتون:

```
enumerate(files)
```

به ما شماره و مقدار را با هم می‌دهد.

مثال:

```
fruits = ["apple", "banana", "orange"]
for index, fruit in enumerate(fruits):
    print(index, fruit)
```

فروچی:

0 apple

1 banana

2 orange

استفاده در پروژه

for index, file in enumerate(files):

print(index + 1, file)

چون آدم‌ها معمولاً شماره‌گذاری را از ۱ دوست دارند.

فروچی:

1 photo.jpg

2 movie.mp4

3 book.pdf

4 Documents

یک نکته مهم

(`listdir()` فقط اسم فایل را برمی‌گرداند).

مثلاً:

photo.jpg

نه:

D:\Downloads\photo.jpg

یعنی مسیر کامل را به ما نمی‌دهد.

بعداً که بفوایم فایل‌ها را جابه‌جا کنیم به مسیر کامل نیاز داریم.

برای همین در قدم بعدی با یکی از مهم‌ترین توابع os آشنا می‌شویم:

```
os.path.join()
```

که مسیرها را به شکل مرفه‌ای به هم وصل می‌کند.

مثلاً:

```
path = "D:\\Downloads"
```

```
file = "photo.jpg"
```

را تبدیل می‌کند به:

```
D:\\Downloads\\photo.jpg
```

و تقریباً در تمام پروژه‌های فایل و پوشه از آن استفاده می‌شود.

نسخه کامل پروژه تا اینجا

```
import os
```

```
def is_valid_folder(path):
```

```
    return os.path.exists(path) and os.path.isdir(path)
```

```
path = input("Enter folder path: ")
```

```
if is_valid_folder(path):
```

```
    files = os.listdir(path)
```

```
    for index, file in enumerate(files):
```

```
        print(index + 1, file)
```

```
else:
```

```
    print("Invalid folder")
```

قبل از رفتن به مرحله بعد، این کد را اجرا کن و ببین چه فروجه می‌گیری. همچنین دقت کن که

آیا پوشه‌های داخل مسیر را هم نمایش می‌دهد یا نه؛ این نکته در مرحله بعدی مهم می‌شود

الان `os.listdir()` هیچ تفاوتی بین فایل و پوشه قائل نمیشه.

مثلاً:

Downloads

└─ photo.jpg

└─ movie.mp4

└─ book.pdf

└─ MyFolder

فروچی:

photo.jpg

movie.mp4

book.pdf

MyFolder

برای `listdir()` همه فقط اسم هستن.

قدم پنجم: ساخت مسیر کامل فایل‌ها

الان متغیر `file` فقط اینو نگه می‌داره:

photo.jpg

اما اگر بخوایم بفهمیم فایل هست یا پوشه، باید مسیر کاملش رو داشته باشیم.

مثلاً:

D:\Downloads\photo.jpg

مشکل اتصال رشته‌ها

میشه اینطوری نوشت:

```
full_path = path + "\\\" + file
```

اما این روش مرفه‌ای نیست.

چون:

- روی ویندوز \ داریم.
- روی لینوکس / داریم.
- ممکنه اشتباه تایپی پیش بیاد.

os.path.join()

برای همین از:

```
os.path.join()
```

استفاده می‌کنیم.

مثال:

```
import os
path = r"D:\Downloads"
file = "photo.jpg"
full_path = os.path.join(path, file)
print(full_path)
```

خروجی:

```
D:\Downloads\photo.jpg
```

چرا اسمش join است؟

چون چند قسمت مسیر را به هم می‌چسباند.

مثال:

```
os.path.join("A", "B", "C")
```

خروجی:

```
A\B\C
```

ملا فایل یا پوشه بودن را تشخیص می‌دهیم

```
full_path = os.path.join(path, file)
print(os.path.isfile(full_path))
```

اگر فایل باشد:

True

اگر پوشه باشد:

False

مثال کامل

```
import os
path = input("Enter folder path: ")
files = os.listdir(path)
for file in files:
    full_path = os.path.join(path, file)
    if os.path.isfile(full_path):
        print(file, "=> FILE")
    else:
        print(file, "=> FOLDER")
```

اگر پوشه این شکلی باشد:

photo.jpg

movie.mp4

Documents

Music

فروبی:

photo.jpg => FILE

movie.mp4 => FILE

Documents => FOLDER

Music => FOLDER

یک نکته مهم درباره متغیرها

اینجا:

for file in files:

متغیر file فقط اسم فایل را دارد.

اما:

full_path

مسیر کامل را دارد.

این دو را قاطی نکنی، چون بعداً موقع جابه‌جایی فایل‌ها خیلی مهم می‌شود.

نسخه کامل پروژه تا اینجا

import os

def is_valid_folder(path):


```
return os.path.exists(path) and os.path.isdir(path)
path = input("Enter folder path: ")
if is_valid_folder(path):
    files = os.listdir(path)
    for index, file in enumerate(files):
        full_path = os.path.join(path, file)
        if os.path.isfile(full_path):
            print(f'{index + 1}. {file} => FILE')
        else:
            print(f'{index + 1}. {file} => FOLDER')
else:
    print("Invalid folder")
```

قدم ششم: تشخیص نوع فایل

پسوند فایل (Extension)

به قسمت آخر اسم فایل می‌گن پسوند.

مثلاً:

photo.jpg

پسوند:

jpg

این پسوند به ما می‌گه فایل چه نوعیه.

روش اول()endswith :

پایتون یک تابع برای رشته‌ها دارد:

```
text.endswith()
```

مثال:

```
file = "photo.jpg"  
print(file.endswith(".jpg"))
```

فروبی:

True

مثال دوم:

```
file = "photo.jpg"  
print(file.endswith(".pdf"))
```

فروبی:

False

استفاده در پروژه

```
if file.endswith(".jpg"):  
    print("Image")
```

چند نوع فایل

```
if file.endswith(".jpg"):  
    print("Image")  
elif file.endswith(".pdf"):  
    print("Document")
```

```
elif file.endswith(".mp4"):
```

```
    print("Video")
```

```
else:
```

```
    print("Unknown")
```

مشکل

اگر فایل این باشد:

PHOTO.JPG

یا:

Photo.Jpg

کد ما فراب می‌شود. چون:

"JPG" != ".jpg"

lower()

برای حل این مشکل:

file.lower()

همه مروف را کوچک می‌کند.

مثال:

```
print("PHOTO.JPG".lower())
```

خروجی:

photo.jpg

پس بهتر است:

```
if file.lower().endswith(".jpg"):
```

یک قابلیت جالب `endswith`

می‌تواند چند پسوند را با هم بگیرد:

```
file.lower().endswith(  
    ".jpg", ".jpeg", ".png"  
)
```

اگر هرکدام باشد `True` برمی‌گرداند.

دسته‌بندی فایل‌ها

می‌توانیم بگوییم:

تصاویر

```
("jpg", "jpeg", "png", "gif")
```

ویدیوها

```
("mp4", "avi", "mkv")
```

اسناد

```
("pdf", "docx", "txt")
```

صدا

```
("mp3", "wav")
```

نسخه پروژه

داخل ملکه:

```
if os.path.isfile(full_path):
```

```
if file.lower().endswith(
    ".jpg", ".jpeg", ".png", ".gif")
):
    print(file, "=> IMAGE")
elif file.lower().endswith(
    ".mp4", ".avi", ".mkv")
):
    print(file, "=> VIDEO")
elif file.lower().endswith(
    ".pdf", ".docx", ".txt")
):
    print(file, "=> DOCUMENT")
elif file.lower().endswith(
    ".mp3", ".wav")
):
    print(file, "=> AUDIO")
else:
    print(file, "=> UNKNOWN")
```

اما یک مشکل طرایی داریم

اگر بفوایم ۲۰ نوع فایل اضافه کنیم، کد فیلی شلوغ می‌شود.

برنامه‌نویس مرفه‌ای معمولاً این کار را داخل یک تابع انجام می‌دهد.

مثلاً:

```
def get_file_type(filename):
```

...

و بعد:

```
file_type = get_file_type(file)
print(file_type)
```

این کد فیللی تمیزتر است.

نسخه کامل پروژه تا اینجا

```
import os
def is_valid_folder(path):
    return os.path.exists(path) and os.path.isdir(path)
path = input("Enter folder path: ")
if is_valid_folder(path):
    files = os.listdir(path)
    for index, file in enumerate(files):
        full_path = os.path.join(path, file)
        if os.path.isfile(full_path):
            if file.lower().endswith(
                (".jpg", ".jpeg", ".png", ".gif")
            ):
                print(f"{index + 1}. {file} => IMAGE")
            elif file.lower().endswith(
                (".mp4", ".avi", ".mkv")
            ):
                print(f"{index + 1}. {file} => VIDEO")
```

```
elif file.lower().endswith(
    ".pdf", ".docx", ".txt")
):
    print(f"{index + 1}. {file} => DOCUMENT")
elif file.lower().endswith(
    ".mp3", ".wav")
):
    print(f"{index + 1}. {file} => AUDIO")
else:
    print(f"{index + 1}. {file} => UNKNOWN")
else:
    print(f"{index + 1}. {file} => FOLDER")
else:
    print("Invalid folder")
```

قدم هفتم: سافت پوشه‌های مورد نیاز

فرض کن داخل Downloads این فایل‌ها رو داری:

photo.jpg
movie.mp4
book.pdf
song.mp3

ما می‌فوییم آخر کار این ساختار رو داشته باشیم:

Downloads
└─ Images
└─ Videos

└— Documents

└— Audio

|

└— photo.jpg

└— movie.mp4

└— book.pdf

└— song.mp3

فعلًا فقط پوشه‌ها (و می‌سازیم، هنوز فایل‌ها جابه‌جا نمی‌کنیم).

os.makedirs()

برای ساخت پوشه از این تابع استفاده می‌کنیم:

os.makedirs("Images")

اگر پوشه وجود نداشته باشد:

Images

ساخته می‌شود.

مشکل

اگر دوباره برنامه رو اجرا کنی:

os.makedirs("Images")

خطا میدهد:

FileExistsError

چون پوشه از قبل وجود دارد.

راه حل

```
os.makedirs("Images", exist_ok=True)
```

exist_ok یعنی چی؟

```
exist_ok=True
```

اگر پوشه از قبل وجود داشت، مشکلی نیست.

پارامتر (Parameter)

تا الان توابعی مثل:

```
print()
```

```
input()
```

```
os.listdir()
```

دیدیم.

بعضی توابع پارامتر می‌گیرند:

مثلاً:

```
print("Hello")
```

پارامتر:

```
"Hello"
```

اینجا هم:

```
os.makedirs("Images", exist_ok=True)
```

دو پارامتر داریم.

پارامتر نامدار (Keyword Argument)

این:

```
exist_ok=True
```

یک پارامتر نامدار هست.

اسمش رو مشخص کردیم.

مثلاً:

```
print("Hello", end="***")
```

هم از همین نوعه.

برای پروژه

بیایم پوشه‌ها رو بسازیم:

```
os.makedirs("Images", exist_ok=True)
os.makedirs("Videos", exist_ok=True)
os.makedirs("Documents", exist_ok=True)
os.makedirs("Audio", exist_ok=True)
```

اما یک مشکل

الان این پوشه‌ها کنار فایل پایتون ساخته میشن.

مثلاً:

Project

└─ main.py

└─ Images

└─ Videos

در حالی که ما می‌خواهیم داخل پوشه‌ای که کاربر انتخاب کرده ساخته بشن.

استفاده از join

مثلاً:

```
images_folder = os.path.join(  
    path,  
    "Images"  
)
```

اگر:

```
path = "D:\\Downloads"
```

باشد:

```
images_folder
```

میشه:

```
D:\\Downloads\\Images
```

حالا:

```
os.makedirs(images_folder, exist_ok=True)
```

ساخت همه پوشه‌ها

```
os.makedirs(  
    os.path.join(path, "Images"),
```

```
        exist_ok=True
    )
    os.makedirs(
        os.path.join(path, "Videos"),
        exist_ok=True
    )
    os.makedirs(
        os.path.join(path, "Documents"),
        exist_ok=True
    )
    os.makedirs(
        os.path.join(path, "Audio"),
        exist_ok=True
    )
```

مرفه‌ای‌تر

چون داریم یک کار مشابه (و چند بار تکرار می‌کنیم):

```
folders = [
    "Images",
    "Videos",
    "Documents",
    "Audio"
]
```

بعد:

```
for folder in folders:
```

```
    folder_path = os.path.join(
```

```
        path,
```

```
        folder
```

```
    )
```

```
    os.makedirs(
```

```
        folder_path,
```

```
        exist_ok=True
```

```
    )
```

نسخه کامل پروژه تا اینجا

```
import os
```

```
def is_valid_folder(path):
```

```
    return os.path.exists(path) and os.path.isdir(path)
```

```
path = input("Enter folder path: ")
```

```
if is_valid_folder(path):
```

```
    folders = [
```

```
        "Images",
```

```
        "Videos",
```

```
        "Documents",
```

```
        "Audio"
```

```
    ]
```

```
for folder in folders:
```

```
    folder_path = os.path.join(
```

```
        path,
```

```
        folder
```

```
)  
os.makedirs(  
    folder_path,  
    exist_ok=True  
)  
files = os.listdir(path)  
for index, file in enumerate(files):  
    full_path = os.path.join(  
        path,  
        file  
    )  
  
    if os.path.isfile(full_path):  
        if file.lower().endswith(  
            (".jpg", ".jpeg", ".png", ".gif")  
        ):  
            print(f"{index + 1}. {file} => IMAGE")  
        elif file.lower().endswith(  
            (".mp4", ".avi", ".mkv")  
        ):  
            print(f"{index + 1}. {file} => VIDEO")  
        elif file.lower().endswith(  
            (".pdf", ".docx", ".txt")  
        ):  
            print(f"{index + 1}. {file} => DOCUMENT")
```

```
elif file.lower().endswith(
    ".mp3", ".wav")
):
    print(f"{index + 1}. {file} => AUDIO")
else:
    print(f"{index + 1}. {file} => UNKNOWN")
else:
    print("Invalid folder")
```

قدم هشتم: جابه‌جا کردن فایل‌ها

ماژول shutil :

تا الان از os استفاده می‌کردیم.

برای عملیات روی فایل‌ها (کپی، انتقال، حذف و ...) معمولاً از ماژول shutil استفاده می‌کنیم.

ابتدای برنامه:

```
import shutil
```

تابع move()

مهم‌ترین تابع فعلاً:

```
shutil.move(source, destination)
```

یعنی:

فایل را از source به destination منتقل کن.

فرض کن:

D:\Downloads\photo.jpg

وجود دارد.

می‌نویسیم:

```
shutil.move(  
    r"D:\Downloads\photo.jpg",  
    r"D:\Downloads\Images\photo.jpg"  
)
```

بعد از اجرا:

D:\Downloads\Images\photo.jpg

وجود دارد و فایل از محل قبلی حذف می‌شود.

Destination و Source

فایلی مهمه قاطی نکنی.

source

مسیر فعلی فایل

destination

مسیر جدید فایل

استفاده در پروژه

ما قبلاً داشتیم:

```
full_path = os.path.join(path, file)
```


مثلاً:

D:\Downloads\photo.jpg

این همیشه Source.

حالا مقصد رو می‌سازیم:

```
destination = os.path.join(  
    path,  
    "Images",  
    file  
)
```

نتیجه:

D:\Downloads\Images\photo.jpg

و انتقال:

```
shutil.move(  
    full_path,  
    destination  
)
```

اولین انتقال واقعی

داخل بفش تصاویر:

```
if file.lower().endswith(  
    ".jpg", ".jpeg", ".png", ".gif"  
):
```

```
destination = os.path.join(
    path,
    "Images",
    file
)
shutil.move(
    full_path,
    destination
)
print(file, "moved to Images")
```

نکته مهم

وقتی فایل جابه‌جا میشه:

shutil.move(...) دیگه در محل قبلی وجود نداره. پس نباید دوباره بخوای از همون مسیر استفاده کنی.

برای همه دسته‌ها

می‌تونیم بنویسیم:

if image:

-> Images

elif video:

-> Videos

elif document:

-> Documents

elif audio:

-> Audio

یک متغیر جدید

به جای اینکه چهار بار کد تکراری بنویسیم:

target_folder = "Images"

یا:

target_folder = "Videos"

و در آخر فقط یک بار:

destination = os.path.join(

path,

target_folder,

file

)

shutil.move(

full_path,

destination

)

شمارش فایل‌های منتقل شده

قبل از حلقه:

moved_files = 0

بعد از هر انتقال:

moved_files += 1

+= یعنی چی؟

این:

```
x += 1
```

معادل:

```
x = x + 1
```

است.

مثال:

```
count = 0
```

```
count += 1
```

```
print(count)
```

خروجی:

```
1
```

در پایان:

```
print(
```

```
    f"\nMoved {moved_files} files"
```

```
)
```

f-string چیست؟

مثلاً:

```
name = "Narges"
```

```
print(f"Hello {name}")
```

فروبی:

Hello Narges

نسخه کامل پروژه تا اینجا

```
import os
import shutil
def is_valid_folder(path):
    return os.path.exists(path) and os.path.isdir(path)
path = input("Enter folder path: ")
if is_valid_folder(path):
    folders = [
        "Images",
        "Videos",
        "Documents",
        "Audio"
    ]
    for folder in folders:
        folder_path = os.path.join(
            path,
            folder
        )
        os.makedirs(
            folder_path,
            exist_ok=True
```

```
)  
moved_files = 0  
files = os.listdir(path)  
for file in files:  
    full_path = os.path.join(  
        path,  
        file  
    )  
    if not os.path.isfile(full_path):  
        continue  
    target_folder = None  
    if file.lower().endswith(  
        (".jpg", ".jpeg", ".png", ".gif")  
    ):  
        target_folder = "Images"  
    elif file.lower().endswith(  
        (".mp4", ".avi", ".mkv")  
    ):  
        target_folder = "Videos"  
    elif file.lower().endswith(  
        (".pdf", ".docx", ".txt")  
    ):  
        target_folder = "Documents"  
    elif file.lower().endswith(  
        (".mp3", ".wav")  
    ):
```

```
target_folder = "Audio"
if target_folder:
    destination = os.path.join(
        path,
        target_folder,
        file
    )
    shutil.move(
        full_path,
        destination
    )
    moved_files += 1
    print(
        f"{file} -> {target_folder}"
    )
    print(
        f"\nMoved {moved_files} files."
    )
else:
    print("Invalid folder")
```

نکته مهم

قبل از اجرای این نسخه، روی یک پوشه آزمایشی تستش کن، نه روی Downloads اصلی. چون این نسخه واقعاً فایل‌ها را جابه‌جا می‌کند.

الان می‌رسیم به چیزی که خیلی از تازه‌کارها یاد نمی‌گیرن ولی برنامه‌نویس‌های حرفه‌ای دائماً انجام میدن:

Refactoring بازآرایی کد

کد رو بدون تغییر دادن رفتار برنامه، تمیزتر و قابل نگهداری‌تر کنیم.

الان این قسمت رو ببین:

```
if file.lower().endswith(
    ".jpg", ".jpeg", ".png", ".gif")
):
    target_folder = "Images"
elif file.lower().endswith(
    ".mp4", ".avi", ".mkv")
):
    target_folder = "Videos"
elif file.lower().endswith(
    ".pdf", ".docx", ".txt")
):
    target_folder = "Documents"
elif file.lower().endswith(
    ".mp3", ".wav")
):
    target_folder = "Audio"
```

کار می‌کنه.

اما تصور کن بخواه:

- zip
- rar
- exe
- py
- html
- css
- js

و ۵۰ نوع فایل دیگر اضافه کنی.

کد فیلی شلوغ میشه.

ساخت تابع `get_file_type`

ایده:

```
file_type = get_file_type(file)
```

و تابع خودش تشخیص بده.

نسخه اول

```
def get_file_type(filename):  
    if filename.lower().endswith(  
        ".jpg", ".jpeg", ".png", ".gif"  
    ):  
        return "Images"  
    elif filename.lower().endswith(  
        ".mp4", ".avi", ".mkv"  
    ):
```

```
return "Videos"

elif filename.lower().endswith(
    ".pdf", ".docx", ".txt"
):
    return "Documents"

elif filename.lower().endswith(
    ".mp3", ".wav"
):
    return "Audio"

return None
```

چرا آفرش return None داریم؟

اگر فایل مثلاً این باشه:

setup.exe

هیچ شرطی درست نیست.

پس:

None

برمی‌گرده.

None چیست؟

یک مقدار فاص در پایتون. یعنی هیچ مقداری وجود ندارد.

مثال:

x = None

استفاده

قبلاً داشتیم:

```
target_folder = None
```

```
if ...
```

مثلاً:

```
target_folder = get_file_type(file)
```

فقط یک خط

مفهوم مهم Abstraction :

قبلاً:

```
if endswith(...)
```

```
elif endswith(...)
```

همه جزئیات توی حلقه بود.

الان:

```
target_folder = get_file_type(file)
```

فقط میگیریم:

نوع فایل رو پیدا کن.

و مهم نیست داخل تابع چطوری انجام شده.

به این میگن. Abstraction.

مرفه‌ای‌ترش کنیم

متی تابع بالا هم تکراریه.

برنامه‌نویس مرفه‌ای معمولاً از Dictionary استفاده می‌کند.

Dictionary

قبلاً List داشتیم:

```
fruits = [  
    "apple",  
    "banana"  
]
```

Dictionary:

```
person = {  
    "name": "Narges",  
    "age": 21  
}
```

دسترسی:

```
print(person["name"])
```

خروجی:

Narges

استفاده در پروژه

```
FILE_TYPES = {
```

```
    "Images": (
```

```
        ".jpg",
```

```
        ".jpeg",
```

```
    ".png",
    ".gif"
),
"Videos": (
    ".mp4",
    ".avi",
    ".mkv"
),
"Documents": (
    ".pdf",
    ".docx",
    ".txt"
),
"Audio": (
    ".mp3",
    ".wav"
)
}
```

مثالا:

```
def get_file_type(filename):
    filename = filename.lower()
    for folder, extensions in FILE_TYPES.items():
        if filename.endswith(extensions):
            return folder
    return None
```

items() چیست؟

FILE_TYPES.items()

هر بار دو مقدار میدهد:

مثلاً:

دور اول:

folder = "Images"

extensions = (

"jpg",

"jpeg",

"png",

"gif"

)

دور دوم:

folder = "Videos"

و ...

نسخه کامل پروژه تا اینجا

import os

import shutil

FILE_TYPES = {

"Images": (

"jpg",

```
        ".jpeg",
        ".png",
        ".gif"
    ),
    "Videos": (
        ".mp4",
        ".avi",
        ".mkv"
    ),
    "Documents": (
        ".pdf",
        ".docx",
        ".txt"
    ),
    "Audio": (
        ".mp3",
        ".wav"
    )
}

def is_valid_folder(path):
    return os.path.exists(path) and os.path.isdir(path)

def get_file_type(filename):
    filename = filename.lower()
    for folder, extensions in FILE_TYPES.items():
        if filename.endswith(extensions):
            return folder
```

```
    return None

path = input("Enter folder path: ")

if is_valid_folder(path):
    for folder in FILE_TYPES.keys():
        os.makedirs(
            os.path.join(path, folder),
            exist_ok=True
        )
    moved_files = 0
    files = os.listdir(path)
    for file in files:
        full_path = os.path.join(
            path,
            file
        )
        if not os.path.isfile(full_path):
            continue
        target_folder = get_file_type(file)
        if target_folder:
            destination = os.path.join(
                path,
                target_folder,
                file
            )
            shutil.move(
                full_path,
```



```
        destination
    )
    moved_files += 1
    print(
        f"{file} -> {target_folder}"
    )
    print(
        f"\nMoved {moved_files} files."
    )
else:
    print("Invalid folder")
```

مدیریت خطاها (Exception Handling)

تا الان فرض کردیم همه چیز همیشه درست پیش میره.

اما در دنیای واقعی:

- کاربر مسیر اشتباه میدهد.
- فایل قفل شده.
- مجوز دسترسی وجود نداره.
- اسم فایل مقصد از قبل وجود داره.
- هارد پر شده.

و هزار مشکل دیگه.

اگر برای این موارد آماده نباشی، برنامه کرش میکنه.

Exception چیست؟

مثلاً:

```
number = int("hello")
```

پایتون نمی‌تونه "hello" رو به عدد تبدیل کنه.

خطا:

ValueError

اینجا یک Exception رخ داده.

try / except

برای گرفتن خطاها

try:

```
number = int("hello")
```

except:

```
print("Something went wrong")
```

فروجهی:

Something went wrong

برنامه هم نمی‌بندد.

try دقیقاً یعنی چی؟

try:

یعنی:

این کد رو اجرا کن. اگر خطایی رخ داد، به جای کرش کردن برو بفش. except.

مثال ساده

try:

```
result = 10 / 0
```

except:

```
print("Cannot divide by zero")
```

خروجی:

Cannot divide by zero

گرفتن نوع خطا

بهتره بدونیم چه خطایی رخ داده.

try:

```
result = 10 / 0
```

except Exception as e:

```
print(e)
```

خروجی:

division by zero

Exception چیست؟

تقریباً همه خطاهای رایج از کلاس Exception ارث‌بری می‌کنن.

فعلاً همین رو بدون کافیه.

بعداً در OOP کامل درکش می‌کنی.

استفاده در پروژه

این بخش رو ببین:

```
shutil.move(  
    full_path,  
    destination  
)
```

اگر خطا بده کل برنامه متوقف میشه.

پس می نویسیم:

```
try:  
    shutil.move(  
        full_path,  
        destination  
    )  
except Exception as e:  
    print(  
        f"Error moving {file}: {e}"  
    )
```

چرا بهتره؟

فرض کن ۱۰۰۰ فایل داری. یکی مشکل داشته باشه. اگر try نداشته باشی فایل شماره ۳۵ خطا میده برنامه بسته میشه اما با try:

فایل ۳۵ خطا داد

فایل ۳۶ ...

فایل ۳۷ ...

فایل ۳۸...

برنامه ادامه پیدا می‌کند.

continue

یادت هست قبلاً داشتیم:

continue

یعنی:

این دور ملقه رو رد کن و برو سراغ بعدی.

می‌تونیم داخل except هم استفاده کنیم:

except Exception as e:

print(e)

continue

else بلوک

کمتر شناخته شده ولی خیلی مفیده:

try:

...

except:

...

else:

...

مثال:

try:

```
number = int("10")
```

except:

```
print("Error")
```

else:

```
print("Success")
```

خروجی:

Success

چون خطایی رخ نداده.

بلوک finally

try:

...

except:

...

finally:

...

این قسمت همیشه اجرا می‌شود.

مثلاً:

try:

```
print("Start")
```

finally:

```
print("End")
```

فروچی:

Start

End

در پروژه‌های فایل و دیتابیس خیلی استفاده میشه

نسخه بهتر ملکه انتقال

try:

```
shutil.move(  
    full_path,  
    destination  
)
```

except Exception as e:

```
print(  
    f"Error moving {file}: {e}"  
)
```

else:

```
moved_files += 1  
print(  
    f"{file} -> {target_folder}"  
)
```

یک نکته طراحی

الان همه کدها توی بدنه اصلی برنامه هستن.

برنامه‌نویس مرفه‌ای معمولاً منطق اصلی رو داخل تابع می‌ذاره.

مثلاً:

```
def organize_files(path):
```

```
    ...
```

بعد:

```
organize_files(path)
```

این باعث میشه:

- تست کردن راحت‌تر بشه.
- کد تمیزتر بشه.
- قابلیت توسعه بیشتر بشه.

نسخه کامل پروژه تا اینجا

(فقط بفش جدید رو اضافه می‌کنیم)

```
try:
```

```
    shutil.move(  
        full_path,  
        destination  
    )
```

```
except Exception as e:
```

```
    print(  
        f"Error moving {file}: {e}"  
    )
```


else:

```
moved_files += 1
```

```
print(
```

```
f"{file} -> {target_folder}"
```

```
)
```

پیشنهاد من برای قدم بعدی

قبل از اضافه کردن امکانات جدید، یک بار خودت کل پروژه فعلی رو از اول تا آخر بخون و سعی کن برای هر خط توضیح بدی:

- این خط چه کاری می‌کنه؟

- چرا اینجا گذاشته شده؟

- اگر حذفش کنم چه اتفاقی می‌افته؟

اگر روی ۸۰٪ خطوط بتونی توضیح بدی، یعنی واقعاً یاد گرفتی، نه اینکه فقط پروژه رو جلو برده باشی.

عالی. الان وقتشه پروژه رو از حالت «یه اسکریپت که کار می‌کنه» به حالت «یه برنامه

ساختاریافته» ببریم.

قدم دهم: تابع `organize_files`

الان بدنه اصلی برنامه خیلی شلوغه. یه برنامه‌نویس باتجربه وقتی اینو ببینه میگه: منطق اصلی برنامه باید داخل تابع باشه.

مشکل فعلی

الان تقریباً همه چیز اینجاست:

```
path = input(...)
if is_valid_folder(path):
    ...
    for file in files:
        ...
```

اگر بعداً بخوای:

- رابط گرافیکی بسازی
- بات تلگرام بسازی
- برنامه رو توسعه بدی

کدت به هم ریخته میشه.

ساخت تابع

می‌سازیم:

```
def organize_files(path):
    ...
```

مزیت

قبلاً:

```
if is_valid_folder(path):
    ...

الان:

organize_files(path)
```

فقط یک خط.

انتقال کد

مثلاً:

```
def organize_files(path):  
    for folder in FILE_TYPES.keys():  
        os.makedirs(  
            os.path.join(path, folder),  
            exist_ok=True  
        )  
    moved_files = 0  
    files = os.listdir(path)  
    ...
```

همه منطق انتقال فایل‌ها میره داخل تابع.

مقدار برگشتی

الان تابع فقط کار انجام می‌ده.

بهتره در آخر تعداد فایل‌های منتقل شده رو برگردونه.

```
return moved_files
```

مثال:

```
count = organize_files(path)  
print(count)
```

چرا return بهتره؟

قبلاً گفتیم:

```
print()
```

فقط نمایش میده.

اما:

```
return
```

نتیجه رو به بقیه برنامه میده.

نسخه مرفه‌ای‌تر

```
count = organize_files(path)
```

```
print(
```

```
    f"\nMoved {count} files."
```

```
)
```

ساختار جدید برنامه

الان برنامه سه بخش داره:

داده‌ها

```
FILE_TYPES = {...}
```

توابع

```
is_valid_folder()
```

```
get_file_type()
```

organize_files()

اجرای برنامه

path = input(...)

if is_valid_folder(path):

count = organize_files(path)

else:

print(...)

این سبک اسم داره

بهش میگن:

Separation of Concerns

یعنی:

هر بخش فقط مسئول یک کار باشه.

مثال:

is_valid_folder()

فقط اعتبارسنجی.

get_file_type()

فقط تشخیص نوع فایل.

organize_files()

نسخه کامل پروژه تا اینجا

```
import os
import shutil
FILE_TYPES = {
    "Images": (
        ".jpg",
        ".jpeg",
        ".png",
        ".gif"
    ),
    "Videos": (
        ".mp4",
        ".avi",
        ".mkv"
    ),
    "Documents": (
        ".pdf",
        ".docx",
        ".txt"
    ),
    "Audio": (
        ".mp3",
        ".wav"
    )
}
```

```
}  
  
def is_valid_folder(path):  
    return os.path.exists(path) and os.path.isdir(path)  
  
def get_file_type(filename):  
    filename = filename.lower()  
    for folder, extensions in FILE_TYPES.items():  
        if filename.endswith(extensions):  
            return folder  
  
    return None  
  
def organize_files(path):  
    for folder in FILE_TYPES.keys():  
        os.makedirs(  
            os.path.join(path, folder),  
            exist_ok=True  
        )  
  
    moved_files = 0  
    files = os.listdir(path)  
    for file in files:  
        full_path = os.path.join(  
            path,  
            file  
        )  
  
        if not os.path.isfile(full_path):  
            continue  
  
        target_folder = get_file_type(file)
```

```
if not target_folder:
    continue
destination = os.path.join(
    path,
    target_folder,
    file
)
try:
    shutil.move(
        full_path,
        destination
    )
except Exception as e:
    print(
        f"Error moving {file}: {e}"
    )
else:
    moved_files += 1
    print(
        f"{file} -> {target_folder}"
    )
return moved_files

path = input(
    "Enter folder path: "
)
```



```
if is_valid_folder(path):
    count = organize_files(path)
    print(
        f"\nMoved {count} files."
    )
else:
    print(
        "Invalid folder"
    )
```

ملقه while

تا الان فقط for یاد گرفتیم.

for وقتی فوبه که بدونیم چند بار می‌خوایم تکرار کنیم.

مثلاً:

```
for i in range(5):
    print(i)
```

فروبی:

0
1
2
3
4

دقیقاً ۵ بار اجرا میشه.

اما اگر ندونیم چند بار باید تکرار بشه چی؟

مثلاً:

- تا وقتی کاربر Exit نزده
 - تا وقتی رمز درست وارد نشده
 - تا وقتی بازی تموم نشده
- اینجا از while استفاده می‌کنیم.

اولین while

```
while True:  
    print("Hello")
```

مشکل؟

هیچوقت متوقف نمیشه!

چون:

True

همیشه True هست.

به این میگن:

Infinite Loop حلقه بی‌نهایت.

متوقف کردن حلقه

با:

break

مثال:

```
while True:
```

```
    text = input("Write exit: ")
```

```
    if text == "exit":
```

```
        break
```

break دقیقاً چکار می‌کند؟

فرض کن داخل ۱۰۰ تا حلقه باشی.

وقتی break اجرا بشه:

از حلقه فعلی فوراً خارج شو.

ساخت منو

اولین نسخه:

```
while True:
```

```
    print("\n1. Organize Files")
```

```
    print("2. Exit")
```

```
    choice = input("Select option: ")
```

```
    if choice == "1":
```

```
        print("Organizing...")
```

```
    elif choice == "2":
```

```
        break
```

```
    else:
```

```
        print("Invalid option")
```

چرا choice رشته است؟

چون:

input()

همیشه رشته برمی‌گردونه.

مثلاً:

کاربر بنویسه:

1

در واقع:

"1"

ذخیره میشه.

نه:

1

تفاوت مهم

"1"

رشته

1

عدد

پس:

if choice == "1":

درسته.

اما:

```
if choice == 1:
```

اشتباهه.

اضافه کردن پروژه

الان:

```
path = input(...)
```

فقط یک بار اجرا میشه.

می‌خوایم داخل منو قرارش بدیم.

مثلاً:

```
if choice == "1":
```

```
    path = input(
```

```
        "Enter folder path: "
```

```
    )
```

```
    if is_valid_folder(path):
```

```
        count = organize_files(path)
```

```
        print(
```

```
            f"\nMoved {count} files."
```

```
        )
```

```
    else:
```

```
        print(
```

```
            "Invalid folder")
```

مزیت

الان برنامه بعد از مرتب سازی بسته نمیشه.

برمی گرده به منو.

مثلاً:

1. Organize Files

2. Exit

کاربر:

1

مرتب سازی انجام میشه.

دوباره:

1. Organize Files

2. Exit

کاربر:

1

یه پوشه دیگه مرتب میشه.

تا وقتی:

2

رو نزنه.

وضعیت پروژه

الان پروژه تبدیل میشه به یک برنامه تعاملی.

این دقیقاً شبیه فیللی از ابزارهای خط فرمان (CLI Tools) واقعی هست.

نسخه کامل پروژه تا اینجا

فقط بخش اجرای برنامه رو بایگین کن:

```
while True:
```

```
    print("\n===== FILE ORGANIZER =====")
```

```
    print("1. Organize Files")
```

```
    print("2. Exit")
```

```
    choice = input(
```

```
        "Select option: "
```

```
)
```

```
    if choice == "1":
```

```
        path = input(
```

```
            "Enter folder path: "
```

```
)
```

```
        if is_valid_folder(path):
```

```
            count = organize_files(path)
```

```
            print(
```

```
                f"\nMoved {count} files."
```

```
)
```

```
        else:
```

```
            print(
```

```
                "Invalid folder)
```

```
elif choice == "2":  
    print("Goodbye!")  
    break  
else:  
    print(  
        "Invalid option"  
    )
```

اگر الان به کدت نگاه کنی، می بینی تقریباً تمام مفاهیم پایه پایتون رو لمس کردی:

- متغیر
- شرط
- حلقه
- تابع
- لیست
- دیکشنری
- ماژول
- مدیریت فایل
- مدیریت خطا

این خودش برای یک پروژه مبتدی تا متوسط خیلی خوبه.

الان وارد مرحله ای می شیم که پروژه ها از دید یک آدم عادی، واقعاً شبیه «نرم افزار» می شه.

تا الان برنامه فقط داخل ترمینال کار می کرد:

1. Organize Files

2. Exit

اما اکثر کاربران عادی از پنجره، دکمه و رابط گرافیکی استفاده می‌کنند.

قدم دوازدهم Tkinter :

Tkinter کتابخانه استاندارد سافت رابط گرافیکی در پایتونه.

فوشبفتانه نیاز به نصب نداره و همراه پایتون نصب میشه.

اولین پنجره

```
import tkinter as tk  
root = tk.Tk()  
root.mainloop()
```

خط اول

```
import tkinter as tk
```

داریم کتابخانه رو وارد می‌کنیم.

بهش اسم کوتاه‌تر میدیم:

```
tk
```

که مجبور نباشیم هر بار بنویسیم:

```
tkinter.Tk()
```

سافت پنجره

```
root = tk.Tk()
```

این فط یک پنجره می‌سازد.

بیش می‌گن Root Window پنجره اصلی برنامه.

mainloop

root.mainloop()

این یکی خیلی مهمه.

اگر مذفش کنی: پنجره باز میشه و سریع بسته میشه.

چون:

mainloop()

به پایتون می‌گه:

منتظر تعامل کاربر بمون.

تغییر عنوان پنجره

root.title("File Organizer")

مثال:

import tkinter as tk

root = tk.Tk()

root.title("File Organizer")

root.mainloop()

تغییر اندازه پنجره

```
root.geometry("500x300")
```

یعنی:

عرض = ۵۰۰

ارتفاع = ۳۰۰

مثال:

```
import tkinter as tk
root = tk.Tk()
root.title("File Organizer")
root.geometry("500x300")
root.mainloop()
```

اضافه کردن Label

Label یعنی متن ثابت روی صفحه.

مثال:

```
label = tk.Label(
    root,
    text="File Organizer"
)
```

اما هنوز نمایش داده نمیشه.

باید جایگذاری کنیم.

pack()

```
label.pack()
```

کل کد:

```
import tkinter as tk
root = tk.Tk()
label = tk.Label(
    root,
    text="File Organizer"
)
label.pack()
root.mainloop()
```

Widget چیست؟

در: Tkinter:

- Button
- Label
- TextBox

همه Widget هستن.

اولین Button

```
button = tk.Button(
    root,
    text="Click Me"
)
```

نمایش:

```
button.pack()
```

کد:

```
import tkinter as tk
root = tk.Tk()
button = tk.Button(
    root,
    text="Click Me"
)
button.pack()
root.mainloop()
```

وقتی دکمه کلیک شد

اینجا وارد مفهوم Event میشیم.

تابع:

```
def hello():
    print("Hello")
```

اتصال به دکمه:

```
button = tk.Button(
    root,
```

```
text="Click Me",  
command=hello  
)
```

نکته مهم:

درسته:

```
command=hello
```

اشتباه:

```
command=hello()
```

چون:

```
hello()
```

همون لحظه اجرا میشه.

اولین GUI واقعی

```
import tkinter as tk  
def hello():  
    print("Hello")  
root = tk.Tk()  
root.title("File Organizer")  
root.geometry("400x200")  
label = tk.Label(  
    root,
```

```
text="Welcome to File Organizer"
)
label.pack()
button = tk.Button(
    root,
    text="Click Me",
    command=hello
)
button.pack()
root.mainloop()
```

ارتباط با پروژه ما

هدف نهایی:

به جای این:

```
path = input(
    "Enter folder path: ")
```

کاربر روی دکمه بزنه.

یک پنجره انتخاب پوشه باز بشه.

پوشه رو انتخاب کنه.

برنامه فایل‌ها رو مرتب کنه.

برای این کار در مرحله بعد با:

```
from tkinter import filedialog
```

آشنا میشی.

که می‌تونه پنجره انتخاب پوشه ویندوز رو باز کنه.

نسخه کامل پروژه تا اینجا

فعلاً پروژه اصلی فایل‌ها رو دست نمی‌زنیم.

فقط این فایل جدید رو جداگانه بساز:

```
import tkinter as tk
def hello():
    print("Hello")
root = tk.Tk()
root.title("File Organizer")
root.geometry("400x200")
label = tk.Label(
    root,
    text="Welcome to File Organizer"
)
label.pack()
button = tk.Button(
    root,
    text="Click Me",
    command=hello
)
button.pack()
root.mainloop()
```


و باهاش بازی کن:

- متن Label رو عوض کن.

- متن Button رو عوض کن.

- اندازه پنجره رو تغییر بده.

وقتی فهمیدی Widget ها چطوری ساخته میشن، مرحله بعدی رو میریم و برنامه واقعی File Organizer رو به این پنجره وصل می‌کنیم تا با کلیک روی یک دکمه پوشه انتخاب بشه و فایل‌ها مرتب بشن.

قدم سیزدهم: انتخاب پوشه با پنجره ویندوز

تا الان داشتیم:

```
path = input("Enter folder path: ")
```

اما کاربر باید مسیر رو دستی تایپ می‌کرد.

می‌خوایم این اتفاق بیفته:

۱. کاربر روی دکمه کلیک کنه.

۲. پنجره انتخاب پوشه باز بشه.

۳. پوشه رو انتخاب کنه.

۴. برنامه فایل‌ها رو مرتب کنه.

ماژول جدید

```
from tkinter import filedialog
```

filedialog مجموعه‌ای از پنجره‌های آماده ویندوزه:

- انتخاب فایل

- ذخیره فایل

- انتخاب پوشه

انتخاب پوشه

```
folder_path = filedialog.askdirectory()
```

این خط:

- پنجره انتخاب پوشه باز می‌کند.

- مسیر انتخاب شده رو برمی‌گردونه.

مثلاً:

D:\Downloads

تست

```
import tkinter as tk
from tkinter import filedialog
def choose_folder():
    folder_path = filedialog.askdirectory()
    print(folder_path)
root = tk.Tk()
button = tk.Button(
    root,
    text="Choose Folder",
    command=choose_folder)
```

```
button.pack()
root.mainloop()
```

اگر کاربر Cancel زد چی؟

اگر کاربر پنجره رو ببندد:

```
folder_path = ""
```

(شسته خالی)

برمی‌گردد.

پس باید چک کنیم:

```
if not folder_path:
    return
```

not اینجا چه؟

(شسته خالی):

```
""
```

در پایتون False محسوب میشه.

پس:

```
if not folder_path:
```

یعنی:

اگر چیزی انتخاب نشده بود.

اتصال به پروژه

قبلاً داشتیم:

```
count = organize_files(path)
```

مثلاً:

```
count = organize_files(folder_path)
```

نمایش نتیجه داخل پنجره

الان نتیجه توی ترمینال چاپ میشه.

مثلاً:

```
print(f"Moved {count} files")
```

اما در GUI بهتره داخل پنجره نمایش بدیم.

Label قابل تغییر

```
result_label = tk.Label(  
    root,  
    text=""  
)  
result_label.pack()
```

بعداً:

```
result_label.config(  
    text=f"Moved {count} files"  
)
```

config چیست؟

تقریباً یعنی:

تنظیمات Widget رو تغییر بده.

مثلاً:

```
label.config(text="Hello")
```

متن Label رو عوض می‌کنه.

نسخه اولیه GUI واقعی

```
import tkinter as tk
from tkinter import filedialog
def choose_folder():
    folder_path = filedialog.askdirectory()
    if not folder_path:
        return
    result_label.config(
        text=f"Selected: {folder_path}"
    )
root = tk.Tk()
root.title("File Organizer")
root.geometry("500x250")
button = tk.Button(
    root,
    text="Choose Folder",
    command=choose_folder
```

```
)  
button.pack(pady=20)  
result_label = tk.Label(  
    root,  
    text=""  
)  
result_label.pack()  
root.mainloop()
```

: pady

```
button.pack(  
    pady=20  
)
```

یعنی:

20 پیکسل فاصله عمودی اضافه کن.

قدم بعدی داخل همین پروژه

به جای:

Selected: ...

می نویسیم:

```
count = organize_files(folder_path)
```

و بعد:

```
result_label.config(  
    text=f"Moved {count} files"
```

)

معماری برنامه الان

Backend

توابع:

is_valid_folder()

get_file_type()

organize_files()

Frontend

Tkinter:

Button

Label

Window

این جداسازی یکی از مهم‌ترین اصول توسعه نرم‌افزاره.

نسخه کامل پروژه تا اینجا

فایل مرتب‌ساز اصلی رو نگه دار.

یک فایل جدید برای GUI بساز و فعلاً این کد رو داخلش تست کن:

```
import tkinter as tk
```

```
from tkinter import filedialog
```

```
def choose_folder():
    folder_path = filedialog.askdirectory()
    if not folder_path:
        return
    result_label.config(
        text=f"Selected: {folder_path}"
    )
root = tk.Tk()
root.title("File Organizer")
root.geometry("500x250")
button = tk.Button(
    root,
    text="Choose Folder",
    command=choose_folder
)
button.pack(pady=20)
result_label = tk.Label(
    root,
    text=""
)
result_label.pack()
root.mainloop()
```

قدم چهاردهم: وصل کردن GUI به موتور برنامه

الان دو بخش جدا داری:

بخش اول

`organize_files(path)`

که فایل‌ها رو مرتب می‌کنه.

بخش دوم

GUI که پوشه رو انتخاب می‌کنه.

الان باید این دو تا رو به هم وصل کنیم.

ساختار درست پروژه

پیشنهاد می‌کنم فایل‌ها رو اینطوری جدا کنی:

FileOrganizer/

|

└─ organizer.py

└─ gui.py

organizer.py

توابع اصلی:

`is_valid_folder()`

`get_file_type()`

`organize_files()`

اینجا قرار می‌گیرن.

gui.py

پنجره و دکمه‌ها

Import فایل خودمون

تا الان:

```
import os
import shutil

می‌کردیم.
```

مثلاً:

```
from organizer import (
    organize_files,
    is_valid_folder
)
```

یعنی:

از فایل organizer.py این توابع رو وارد کن.

تغییر choose_folder

قبلاً:

```
result_label.config(
    text=f"Selected: {folder_path}"
)
```

مثلا:

```
if not is_valid_folder(folder_path):
```

```
    result_label.config(
        text="Invalid folder"
    )
```

```
    return
```

سپس:

```
count = organize_files(
```

```
    folder_path
```

```
)
```

بعد:

```
result_label.config(
```

```
    text=f"Moved {count} files"
```

```
)
```

یک مشکل

فرض کن:

10000 فایل

داخل پوشه باشه.

وقتی:

organize_files(...)

اجرا میشه

ممکنه چند ثانیه طول بکشه.

در این مدت پنجره فریز میشه.

این موضوع رو بعداً با Threading حل می‌کنیم.

فعلاً مشکلی نیست.

نمایش خطاها

اگر خطایی رخ بده:

try:

```
count = organize_files(  
    folder_path  
)
```

except Exception as e:

```
result_label.config(  
    text=f"Error: {e}"  
)
```

نسخه اولیه GUI واقعی

```
import tkinter as tk
```

```
from tkinter import filedialog
```

```
from organizer import (
```

```
    organize_files,
```

```
        is_valid_folder
    )
def choose_folder():
    folder_path = filedialog.askdirectory()
    if not folder_path:
        return
    if not is_valid_folder(folder_path):
        result_label.config(
            text="Invalid folder"
        )
        return
    try:
        count = organize_files(
            folder_path
        )
        result_label.config(
            text=f"Moved {count} files"
        )

    except Exception as e:
        result_label.config(
            text=f"Error: {e}"
        )
root = tk.Tk()
root.title(
    "File Organizer"
```

```
)  
root.geometry(  
    "500x250"  
)  
button = tk.Button(  
    root,  
    text="Choose Folder",  
    command=choose_folder  
)  
button.pack(pady=20)  
result_label = tk.Label(  
    root,  
    text=""  
)  
result_label.pack()  
root.mainloop()
```

الان پروژه واقعاً چه کار می‌کنه؟

کاربر:

۱. برنامه رو باز می‌کنه.
۲. روی دکمه کلیک می‌کنه.
۳. پوشه رو انتخاب می‌کنه.
۴. فایل‌ها مرتب میشن.
۵. نتیجه نمایش داده میشه.

قدم بعدی: ساخت فایل exe

این بخش خیلی مهمه چون:

الان اجرا کردن برنامه نیاز داره:

Python روی سیستم نصب باشه.

برای تبدیل به فایل اجرایی از:

pip install pyinstaller استفاده می‌کنیم.

بعد:

pyinstaller --onefile gui.py

onefile یعنی چی؟

بدون این:

50 فایل

10 پوشه

کلی وابستگی

می‌سازه.

با:

--onefile

فقط:

gui.exe

تمویل می‌گیری.

پوشه خروجی

بعد از ساخت:

dist/

|

└─ gui.exe

این فایل رو می‌تونی روی هر سیستم ویندوزی اجرا کنی حتی اگر پایتون نصب نباشه

ممکن است برای شما هم سوال پیش بیاید (=):

(من تبدیل به فایل اجرایی رو نفهمیدم)

pip install pyinstaller

این رو کجا بزنی؟؟؟

این دستور رو باید داخل ترمینال (Terminal) یا Command Prompt ویندوز بزنی، نه داخل فایل پایتون.

روش ۱: داخل VS Code

اگر با VS Code کار می‌کنی:

۱. از منوی بالا:

۲. Terminal → New Terminal

۳. پایین صفحه یک ترمینال باز میشه.

۴. اونجا بنویس:

```
pip install pyinstaller
```

یا اگر کار نکرد:

```
python -m pip install pyinstaller
```

روش ۲: داخل CMD ویندوز

کلیدهای:

Win + R

رو بزن.

بعد بنویس:

cmd

و Enter بزن.

بعد:

```
pip install pyinstaller
```

از کجا بفهمم نصب شده؟

بزن:

```
pyinstaller --version
```

اگر مثلاً اینو دیدی:

6.15.0

یعنی نصب شده.

بعدش چیکار کنیم؟

فرض کن پروژه‌ات اینجاست:

Desktop

└─ FileOrganizer

└─ gui.py

└─ organizer.py

اول باید وارد پوشه پروژه بشی:

cd Desktop\FileOrganizer

بعد:

pyinstaller --onefile gui.py

بعد از چند ثانیه این پوشه‌ها ساخته میشن:

FileOrganizer

└─ build

└─ dist

└─ gui.spec

└─ gui.py

└─ organizer.py

داخل dist فایل اجرایی هست:

dist

└─ gui.exe

همون فایل نهایی برنامه‌ات.

اگر این خطا رو دیدید:

```
WARNING: Retrying (Retry(total=3, connect=None, read=None, redirect=None, status=None)) after connection broken by 'ConnectTimeoutError(<pip._vendor.urllib3.connection.HTTPSConnection object at 0x00000171DB8AA1A0>, 'Connection to pypi.org timed out. (connect /timeout=15)')': /simple/pyinstaller
```

این خطا رو توی ترمینال دیده

این خطا معمولاً به خاطر مشکل اتصال به اینترنت یا دسترسی به PyPI هست، نه مشکل پایتون یا PyInstaller

باید وی پی ان بزنید دوستان =)

البته بدون این فروجی گرفته هم میشه کار رو پیش برد هیچ مشکلی ایجاد نمیکنه زیاد سرش وقت تلف نکنید =)

گردآوری و تدوین: نرگس باکری

با بهره‌گیری از راهنمایی‌های ChatGPT

پانزده فرورداد ۱۴۰۵

Compiled and Organized by Narges Bakeri.

June 2026